M1 — open questions

Everything unresolved, in one place, with the evidence needed to answer it.

Each item says who owns it, what it blocks, and what happens if we guess. Where there is a sensible default I have said what I would do, so a non-answer still leaves a decision on record rather than a silence.

Verified against the live SOCO cluster and the build on **10 September 2026**.

Summary

Kala answered items 2, 3, 5, 7, 9–12 and 14–18 on 10 September. Those are recorded below and marked ✅. What remains is Aravinth's.

#	Question	Owner	Status
A. Blocking			
1	The `ml_` -scoped write credential	Aravinth	**Open**
2	Which weight table is authoritative	Kala	✅ Moot — conformance descoped
3	Can the delivered scorer be extracted as a pure function	Aravinth	✅ Moot — conformance descoped
4	The `postevents` index	Aravinth	**Open**
B. Assumptions the data has changed			
5	`interestScores` now on 12 of 13 users, not 2 of 12	Kala	✅ Ladder unchanged
6	The collection is `users`, not `registers`	Aravinth	**Open**
7	Live categories are title case	Kala	✅ Confirmed
8	`postevents` has grown, 26 → 50	Kala	✅ Old snapshot, counts have moved
C. Ours to decide — confirm or overrule			
9	The uniqueness key — triple, or plus `model_version`	Kala	✅ Keep `model_version`
10	`categories` plural, not `category`	Kala	✅ Confirmed
11	"Five fields" vs "seven, not five"	Kala	✅ Confirmed
12	A reversal is an `event_type`, not a flag	Kala	✅ Confirmed
13	Two event shapes, one adapter each	Ours	Standing
14	The demo score collection name	Kala	✅ `ml_user_interest_scores_demo`
15	`model_version`, not `version`	Kala	✅ Confirmed
E. Lower priority			
16	DS-007 v1.1 is the authority	Kala	✅ Yes — 27 parameters
17	DS-006 v2, the M1 feature list	Kala	✅ Yes
18	Home & Living merge count — 3 or 4	Kala	✅ Four keys, three merges
19	Is `GET /getInterst` still called	Aravinth	**Open**
20	Has `migrate-interest-scores.js` been run	Aravinth	**Open**

Five items remain, all Aravinth's: 1, 4, 6, 19, 20. Item 1 is the only one that stalls work.

A. Blocking

1. The `ml_`-scoped write credential — Aravinth

Ask: a database user that can write **only** collections prefixed `ml_`, and nothing else. The application credential stays read-only.

Where it stands: the read-only credential is confirmed correct — `readonly_user` holds role `SocoReadonly`, which grants `find`, `listCollections`, `listIndexes`, `collStats`, `dbStats`, `changeStream`, `killCursors`, `planCacheRead` on `soco`. No `insert`, no `update`, no `remove`. That half is done.

The write credential does not exist. The engine currently authenticates with a user holding `atlasAdmin` — full access to every database — which is the opposite of the requirement.

Why it matters: the Python side refuses to write outside `ml_` and 13 tests hold that. But a guard in the client is a guard a future client might not have. The requirement is a guard in the database.

The `db.createRole` / `db.createUser` commands are written out in `HANDOFF.md` section 3. Please also confirm the scoping actually holds — that the user can write `ml_user_interest_scores` and **cannot** write `users`, `posts`, `postevents` or `orders`.

Related: `ml_` must live in the **same database** as the application data (Q14 Option A — the app runs one MongoDB connection and there is no second database). During the build it is on a separate cluster, so nothing has yet been proven against the real deployment shape.

2. Which weight table is authoritative — ✅ moot

> **Kala, 10 September:** conformance is not in scope. Item 3 falls with it.

Nothing now compares Method A against the delivered scorer, so the two tables do not have to be reconciled. **P-06 is simply the weight table**, and it is configuration.

What still stands, because it is about the data rather than about conformance: `comment`, `like` and `save` occur in `postevents` and P-06 has no weight for them, so they score nothing. `click` and `search` have weights and never occur. The engine drops the three unweighted types and counts them, so the figure is visible rather than silent. Only worth revisiting if a demo user is meant to show one of those interactions.

▶ The original question, for the record

3. Can the delivered scorer be extracted as a pure function — ✅ moot

Falls with item 2: there is no conformance comparison to extract it for.

`updateUserInterests` remains un-exported, does its own Mongo read and `$inc`, and swallows every exception in an empty catch. Worth reporting to Aravinth as a finding about that code, but it no longer blocks anything here.

4. The `postevents` index — Aravinth

```
db.postevents.createIndex({ user_id: 1, created_at: -1 });
```

All seven existing indexes lead with `influencer`, `post` or `product`, so reading one user's event history scans the collection. Not blocking on generated data; blocking for anything measured against production shape, including the 200ms latency budget.

B. Assumptions the live data has changed

These were stated as fact in the build documents and are no longer true. They need re-confirming because work depends on them.

5. `interestScores` exists on 12 of 13 users, not 2 of 12 — Kala

Stated: "`interestScores` exists on only 2 of 12 users," and the fallback ladder is built around that — rungs 4 and 5 must fall through to rung 3 and log it, "because the delivered score does not exist for almost anybody."

Found: 13 users, **all 13** carry an `interest` array, and **12** carry `interestScores`. The keys are already lowercase canonical, e.g.

```
{ "interest": ["Fashion & Apparel", "Home & Living"],
  "interestScores": { "fashion & apparel": 30, "shoes & footwear": 0 } }
```

Consequence: the argument for the fall-through no longer holds — the delivered score now exists for nearly everybody.

> **Kala, 10 September:** keep the fall-through. It is not used in the demonstration either way.

Settled: rungs 4 and 5 continue to fall through to rung 3 and log it. The behaviour is unchanged; only the reason for it has weakened.

Bonus — this looks like the answer to item 20. Lowercase canonical keys are `toCategoryKey()` output, which suggests `migrate-interest-scores.js` **has** been run. Still Aravinth's to confirm.

6. The collection is `users`, not `registers` — confirm

The specification says the floor comes from `registerModel.interest`. There is **no** `registers` **collection** on the cluster. The `interest` array is on `users`, on all 13 documents.

I have pointed the engine at `users`. Reading `registers` would have returned nothing without raising, so every declared floor would silently have been zero.

Please confirm `registerModel` is backed by the `users` collection and not something else that has yet to be deployed.

7. Live categories are title case

`posts.categories` holds `["Fashion & Apparel"]`. The contract and the score store require `toCategoryKey()` output — lowercase, trimmed, spaces preserved: `"fashion & apparel"`.

Normalisation therefore happens on read, using `categories.js`.

> **Kala, 10 September:** yes, title case.

Settled. `toCategoryKey()` on read is the single normalisation step.

8. `postevents` has grown — ✅

Stated as 26 documents; now **50**. Users stated as 12; now **13**. `userevents` is still empty, as documented.

> **Kala, 10 September:** the evidence column is a 3 September snapshot; the counts on 26 events and 12 users have moved.

Settled. Any figure quoted from the build documents predates 3 September and should be re-read from the cluster before the demonstration rather than repeated.

C. Decisions that are ours to make

I have taken a position on each so work continues. Each is cheap to reverse **now** and expensive later. Please confirm or overrule.

9. The uniqueness key

DS-015 says "one document per user per category". Task 15 says Method B "writes under its own method value so both methods coexist in one store". Those reconcile only as `user_id + category + method`.

Built as: `user_id + category + method + model_version` — that triple plus the model version.

	Same version re-run	New version
Triple only	overwrites	overwrites — previous generation lost
With `model_version`	overwrites	accumulates a new generation

Why: a parameter change should not destroy the evidence of what the previous configuration produced.

Cost: the store grows per version, and the display script must filter on `model_version` or it shows every generation at once.

> **Kala, 10 September:** keep `model_version`.

Settled. The key is `user_id + category + method + model_version`, as built. The display script must filter on `model_version` so it shows one generation at a time.

10. `categories` is plural and an array — ✅

The task list says `category`, singular. It has to be plural.

A post carrying several categories contributes **full weight to each**, not a divided share — that is U03's entire purpose, and U03 is a narrated demo user. A singular field cannot express it, so the generator would need a second code path just for U03 — a special case built to make a demonstration work, which is a finding rather than a shortcut.

The array holds one element in the ordinary case, so plural costs nothing.

> **Kala, 10 September:** correct — the generator uses `categories`.

11. "Five fields" and "seven, not five" are both right — ✅

Five on every row — `user_id`, `event_type`, `categories`, `timestamp`, `weight` — plus two conditional: `view_duration_seconds` on views, `order_id` on purchases and reversals.

> **Kala, 10 September:** correct. Five unconditional plus two conditional. Matches the generator exactly.

12. A reversal is an `event_type`, not a flag — ✅

`purchase_reversal` is its own row type rather than a boolean on a purchase row. It keeps the contract at seven fields, makes a row self-describing when read in Atlas, and means a reversal cannot be created accidentally by a defaulted flag.

> **Kala, 10 September:** correct, and matches the generator, which emits `purchase_reversal` at weight −20 carrying the original `order_id`.

Settled, and the engine already agrees: `expected_weight()` returns the purchase weight negated, and the contract makes `order_id` mandatory on the row.

13. Two event shapes, one adapter each

Swapping a collection name does not make two documents compatible. A demo row carries `categories`; a `postevent` carries `post_id` and the category is a join away — confirmed, `postevents` has no category field at all.

Chosen: a reader adapter per source producing one internal event shape. The alternative was having the generator also emit posts-shaped data so the live query runs unchanged. The adapter is less work and keeps the generated data readable, but it is a real fork and it should be a decision rather than a discovery.

14. The demo score collection name — ✅

> **Kala, 10 September:** `ml_user_interest_scores_demo`.

Settled, and matches what is built. This is the name that goes into Aravinth's credential scope alongside `ml_user_interest_scores`.

15. `model_version`, not `version` — ✅

The task table says `version`; the build context, ST1 and ST4 all say `model_version`, and it is half the unique index.

> **Kala, 10 September:** accept yours — `model_version`.

D. Smaller things, for the record

BSON dates carry milliseconds, not microseconds. A timestamp written and read back is not equal to the one that went in. Irrelevant to decay at day scale, but it will break any conformance test that compares timestamps exactly.

`posts.userId` **holds both ObjectId and String.** Any join on it must match both forms; a `$lookup` matches one type and returns a partial result — which is worse than an empty one, because it looks like the query worked. Handled, but worth knowing before anyone writes an aggregation.

`session_id` **is** `fv_<postId>_<ts>` — one per view, not per session. It cannot be used to group a user's activity.

Credentials in logs. A connection string reached a test traceback during the build. Both URIs are now marked so they cannot be printed, and a test enforces it. The exposed Atlas password should still be rotated.

E. Answered

16. DS-007 v1.1 is the authority — ✅

> **Kala, 10 September:** yes. Twenty-seven parameters.

One thing to check back. The config layer holds the **13** parameters the M1 build documents name: P-02, P-04 to P-11, P-22, P-25, P-26, P-27. If DS-007 v1.1 carries 27 and the rest belong to other models, nothing is missing. If any of the other 14 apply to M1, they are absent from the config and from `ctx.tunables`. A copy of v1.1 would settle it in a minute.

17. DS-006 v2 — ✅

> **Kala, 10 September:** yes. Note that the evidence column is a 3 September snapshot and the counts on 26 events and 12 users have moved.

Consistent with item 8. Treat every figure in the build documents as a 3 September reading.

18. Home & Living — four keys, three merges — ✅

> **Kala, 10 September:**

```
'home & decor' -> 'Home & Living' genuine merge 'kitchen & dining' -> 'Home & Living' genuine merge 'bedding & bath' -> 'Home & Living' gen
```

Settled. DS-008's "four" counts map entries; three of them are real merges and the fourth is an underscore variant of the canonical label correcting itself.

What U06 must show: three distinct old categories collapsing into one, and separately an underscored spelling being repaired. Presenting all four as merges would overstate what the migration does.

This is also why the event contract rejects underscores in a category key — the underscore form is a legacy spelling to be mapped away, never a value to store.

F. Still open — all Aravinth's

Five items, one message. Only the first stalls work.

#	Ask
1	**The `ml_`-scoped write credential.** Able to write `ml_user_interest_scores` and `ml_user_interest_scores_demo` and nothing else. Commands in `HANDOFF.md` §3. The engine currently runs as a full admin, which is the opposite of the requirement
4	**The index** `db.postevents.createIndex({ user_id: 1, created_at: -1 })`. Every existing index leads with influencer, post or product, so reading one user's history scans
6	**Confirm `registerModel` is backed by the `users` collection.** There is no `registers` collection; `interest` is on `users`, all 13 documents. Reading the wrong name returns nothing without erroring, so every declared floor would silently be zero
19	**Is `GET /getInterst` still called by any client?** It returns a stale 20-item list; six entries have no canonical equivalent and no legacy mapping, so they would become permanently unresolvable floor entries
20	**Has `migrate-interest-scores.js` been run?** The lowercase canonical keys in `interestScores` suggest yes, but execution is a separate question from the script existing, and the answer changes what U06 demonstrates

Two items are now closed as **moot**: conformance is out of scope, so the weight-table ruling (2) and extracting `updateUserInterests` (3) no longer block anything. The `updateUserInterests` defects are still worth reporting to Aravinth as findings about that code — not exported, its own Mongo read and `$inc`, and an empty catch — but nothing here waits on them.

Appendix — what is actually on the cluster

Read-only, `soco` database, 10 September 2026.

Collection	Documents	Note
`postevents`	50	The live event source
`posts`	8	All carry `categories`, title case
`orders`	8	Has `created_at` and `order_status`
`users`	13	All 13 have `interest`; 12 have `interestScores`
`userevents`	0	TTL index never created — do not read
`trackings`	5	No duration field
`registers`	—	**Does not exist**

`postevents` **fields:** `_id`, `created_at`, `event_type`, `influencer_id`, `order_id`, `post_id`, `product_id`, `referral_code`, `session_id`, `user_id`, `view_duration_seconds`, `viewer_city`, `viewer_district`, `viewer_gender`, `viewer_is_business`.

No `categories` field — hence the join through `posts`.

Credentials: `readonly_user` → `SocoReadonly` → read on `soco`, verified no write actions. Engine write user → `atlasAdmin`, unscoped — see item 1.
